CSCI 235 Expression Evaluator (practice only, no need to submit)

Thursday, November 12, 2026

On Tuesday we wrote `tokenize` and the first part of `toPostfix`. Today we finish the pipeline

$$\text{text} \xrightarrow{\texttt{tokenize}} \text{tokens} \xrightarrow{\texttt{toPostfix}} \text{postfix} \xrightarrow{\texttt{evalPostfix}} \text{number}.$$

Think of a railway yard: numbers go straight through, but each operator waits on a side track (the stack) until it is safe to let it out. **Fill in the six TODOs in the code on the next pages.**

Rules to write the TODOs (fill in the blanks)

TODO 1. A `"("` must be _____. Why does `prec("(")` return 0?

_____

TODO 2. On a `")"`: pop operators to _____ until you see _____, then _____ the `"("`. What if the stack becomes empty first?

_____

TODO 3. Any other token should _____.

TODO 4. After the loop, the operators still on the stack must be _____. What if one of them is `"("`?

_____

TODO 5. `evalPostfix`: numbers are _____; an operator pops _____ values (the _____ operand first), and the result is _____.

TODO 6. `evalInfix` (two stacks, no postfix): a number goes on the _____ stack. An operator first applies every waiting operator of _____ or equal precedence, then waits on the _____ stack. On `)`, apply until `(`. At the end, apply _____.

Trace by hand: `12 + 3 * (1 - 2) / 4`

Token	Operator stack (top on right)	Output so far	Action
12			
+			
3			
*			
(			
1			
-			
2			
)			
/			
4			
end			

Practice only, no need to submit. Keep it for your notes.

Key idea: from digit characters to a number

The character `'7'` is not the number 7. Digits are stored in order, so `'7' - '0'` *is* 7. To read `345` one digit at a time, start with `num = 0` and, for each next digit character `ch`, do

```
num = num * 10 + (ch - '0');
```

next `ch`	`ch - '0'`	`num` after the step
`'3'`	3	$0 \cdot 10 + 3 = 3$
`'4'`	4	$3 \cdot 10 + 4 = 34$
`'5'`	5	$34 \cdot 10 + 5 = 345$
`'2'`, `'0'`, `'2'`, `'6'`		

This is how a whole-number string such as `"345"` turns into the number 345. In `evalPostfix` and `evalInfix` we call `std::stod(token)`, a more general library function that also handles decimals and exponents. `tokenize` collects the same digits into `numStr`. (An `int` would overflow on a very long literal.)

Two stacks: evaluate `2 * (3 + 4) - 5` directly (TODO 6)

Token	Value stack	Operator stack	Action
`2`			
`*`			
`(`			
`3`			
`+`			
`4`			
`)`			
`-`			
`5`			
end			

Compare. What is the same as in `toPostfix`? What is different? Project 4 can use this idea.

Predict, then run

Input	Postfix (your prediction)	Value
`2 + 3 * 4`		
`(2 + 3) * 4`		
`8 - 3 - 2`		
`100 / 8 / 5`		
`8 / (4 - 4)`		
`(1 + 2`		

Question. In the operator branch, the loop condition uses `>=`. What goes wrong on `8 - 3 - 2` if it becomes `>`?

```
// File name: expression_demo_live.cpp
// Name: Tong Yi
// Class demo: tokenize -> toPostfix -> evalPostfix (partial version)
//
// Thursday: finish the TODOs in toPostfix, write evalPostfix, then evalInfix.
#include <cctype>
#include <iostream>
#include <queue>
#include <stack>
#include <stdexcept> // std::invalid_argument
#include <string>

std::queue<std::string> tokenize(const std::string& expr);
void print(std::queue<std::string> que);
std::queue<std::string> toPostfix(std::queue<std::string> infix);
int prec(const std::string& op);

int main() {
  std::queue<std::string> infix = tokenize("12 + 4");
  print(infix);

  std::queue<std::string> postfix = toPostfix(infix);
  print(postfix);
  return 0;
}

std::queue<std::string> tokenize(const std::string& expr) {
  std::queue<std::string> que;
  int len = static_cast<int>(expr.length());
  int i = 0;
  while (i < len) {
    char ch = expr[i];

    if (std::isdigit(ch)) {
      std::string numStr;
      while (i < len && std::isdigit(expr[i])) {
        numStr += expr[i];
        i++;
      }
      que.push(numStr);
    }
    else if (ch == '+' || ch == '-' || ch == '*' || ch == '/' ||
             ch == '(' || ch == ')') {
      que.push(std::string(1, ch));
      i++;
    }
```

```
    else if (std::isspace(ch)) {
      i++;
    }
    else {
      throw std::invalid_argument("invalid character in infix expression");
    }
  }

  return que;
}

void print(std::queue<std::string> que) {
  while (!que.empty()) {
    std::cout << "\"" << que.front() << "\"" << " ";
    que.pop();
  }
  std::cout << '\n';
}

int prec(const std::string& op) {
  if (op == "(") {
    return 0;
  }
  else if (op == "+" || op == "-") {
    return 1;
  }
  else if (op == "*" || op == "/") {
    return 2;
  }
  else {
    throw std::invalid_argument("invalid operator");
  }
}

std::queue<std::string> toPostfix(std::queue<std::string> infix) {
  std::queue<std::string> postfix;
  std::stack<std::string> ops; // operators waiting to be added to postfix
  while (!infix.empty()) {
    std::string token = infix.front();
    infix.pop();

    if (std::isdigit(token[0])) {
      postfix.push(token);
    }
    else if (token == "+" || token == "-" || token == "*" || token == "/") {
      while (!ops.empty() && prec(ops.top()) >= prec(token)) {
        postfix.push(ops.top());
        ops.pop();
      }
      ops.push(token);
    }
    else if (token == "(") {
      // TODO 1
    }
    else if (token == ")") {
      // TODO 2
    }
    else {
      // TODO 3: what should happen for any other token?
    }
  }

  // TODO 4: dump the rest of ops

  return postfix;
}

// TODO 5: declare and write
//   double evalPostfix(std::queue<std::string> postfix);
// and call it from main.

// TODO 6: evaluate the infix tokens DIRECTLY, with two stacks (one for
// values, one for operators), and no postfix queue. Declare and write
//   double evalInfix(std::queue<std::string> infix);
// and call it from main.
```